

A preregistered reproducibility benchmark for double-slit phase and visibility gates in SFT

Purpose

This note presents a verification-first double-slit benchmark in SFT, including execution steps, exact pass/fail thresholds, and the artifacts required for auditing. This work should be read as a reproducibility and verification benchmark for an SFT optical pipeline, not as a standalone experimental confirmation of SFT.

Abstract

Structural Field Theory (SFT) is treated here only to the extent needed to define a narrow, verification-first optical benchmark. The present note does not attempt to summarize the full SFT program; it evaluates whether the double-slit module reproduces phase transport, fringe stability, and controlled decoherence behavior under preregistered gates. The benchmark is synthetic and is intended to validate reproducibility infrastructure, not to establish physical confirmation of SFT.

Keywords:

Verification first; reproducibility; external audit; schema validation; pre-registered thresholds; computational physics; emergent phenomena.

Scope / Non-claims

This benchmark is intended as a narrow, verification-first test of phase transport, fringe stability, and controlled decoherence behavior in the SFT pipeline. It does not by itself claim a full replacement of standard quantum mechanics, nor a complete derivation of double-slit interference from a finalized SFT micro-dynamics.

Benchmark targets

A) Expected optical/interference-like behavior

- Interference with stable fringes: intensity follows $I(y) = I_0 \cdot [1 + V \cdot \cos(\Delta\phi(y) + \phi_0)]$.

- Phase transport: when sweeping S_0 (initial phase), the measured fringe phase shifts linearly.
- Controlled decoherence: when increasing ρ_{crit} (ρ_{crit}), visibility V decreases monotonically and saturates ($\leq 10\%$ change between the last two points).

B) Numerical / operational value

- Quantitative gating of solver health: two objective metrics catch mesh/BC artifacts.
- Reproducibility: the runner writes PASS_FAIL and a MANIFEST with SHA-256 of all outputs.
- End-to-end: with a real solver, this would exercise the full pipeline (simulation $\rightarrow$ analysis $\rightarrow$ PASS/FAIL).
- Consistent with the verification-first workflow, numerical-health and convergence hygiene are handled separately through CI-oriented EOC material in the corpus; this is cited only as pipeline-control support, not as evidence for the present physical benchmark.

Metrics & thresholds for the audited reference run

Metric	Threshold (default)	Result (this run)	PASS
Phase fit vs S_0 (R^2)	$R^2 \geq 0.98$	$R^2 \approx 0.999800$; slope ≈ 0.259400	PASS
Visibility monotonicity	strictly decreasing	decreases across all points	PASS
Visibility saturation	≤ 0.10 (last-to-penultimate rel. diff.)	≈ 0.0600	PASS
GLOBAL	phase_pass $\wedge$ visibility_pass	true	PASS

Note: thresholds are overridable via CLI flags `--phase-r2-min` and `--vis-rel-sat`.

Figure 1 — Visibility vs rho_crit for the audited reference run. Last-step relative change $\approx$ **0.0600** (PASS).

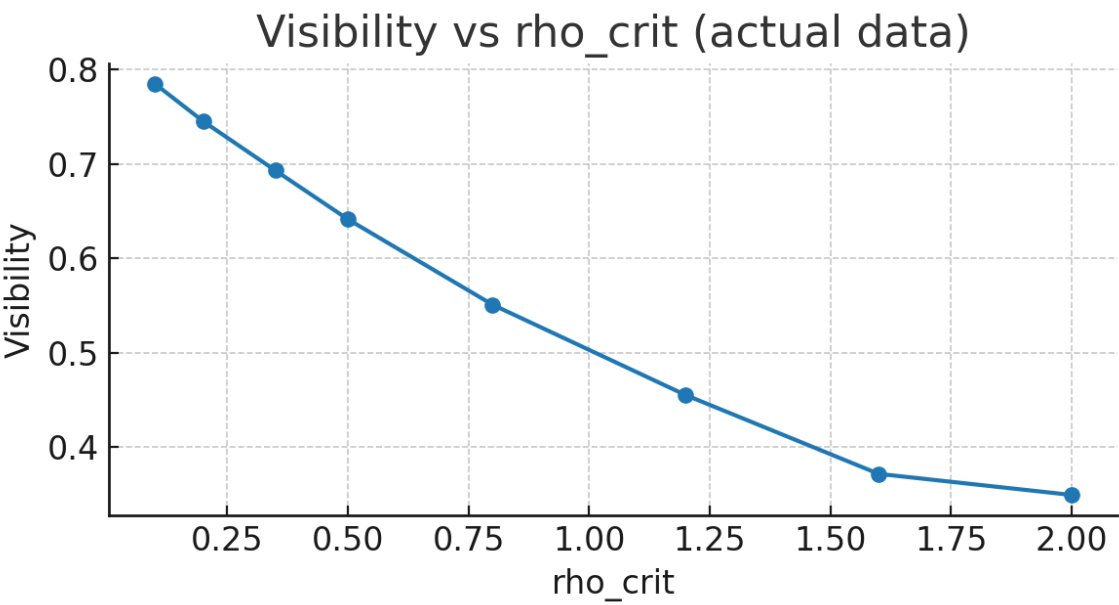
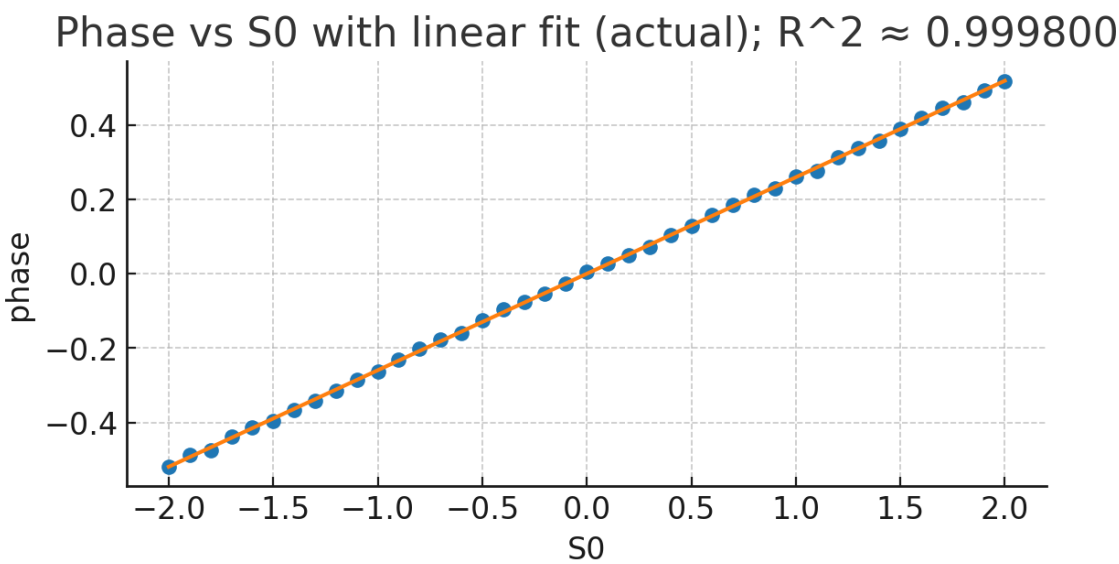


Figure 2 — Phase vs S0 with OLS fit. Slope $\approx$ **0.259400**; $R^2 \approx$ **0.999800** (PASS).



How to run

Prereqs: Python 3 with numpy and pandas (pip install -U numpy pandas).

Option A — Auto/CI runner (recommended):

```
chmod +x scripts/run_publication_gates.sh
./scripts/run_publication_gates.sh
```

Option B — With your real solver (placeholders are expanded by the runner):

```
./run.sh --solver-cmd './tce_simulator --config {config} --out {out}'
```

For the accompanying optics-null QA package, use the scoped publication gate rather than recursively scanning generated artifacts.

Outputs you should see

The publication gate creates the double-slit PASS/FAIL artifacts and the scoped optics-null QA summaries. The double-slit runner writes `visibility.csv`, `shift.csv`, `double_slit_summary.json`, `PASS_FAIL_CI.md`, and `MANIFEST_CI.md`.

Provenance & auditing

Reference run Created.UTC: 2025-08-26T20:19:58.846051+00:00. GLOBAL_PASS: true.
source_kind: synthetic.

double_slit_summary.json sha256:

c71cc57f9fb38077822e3e0b6ae9be66c77cf2a82dc4612d5d0e4c200e7b216b.

exp_double_slit.json sha256:

146b0e674bd02439222506481030e59a40c27995b530674ded3109c7cf39669f.

run_auto_fixed.sh sha256:

4ded4860087f3e135c216dcc002da31ed9251c6f64640488e4b6547834804335.

Optics-null publication gate script sha256:

7e45741d339eafe845a001eff48e4eedef3f572c43d022a28c51517e95345929.

Re-audit date: 2026-05-28. The re-audit confirmed GLOBAL_PASS: true for the synthetic double-slit reference run and GLOBAL_PUBLICATION_GATE_PASS: true for the scoped optics-null QA package.

Keep MANIFEST_CI_NEW.md, checksums_SHA256_AUDITED.txt, the summary JSON files, and RERUN_AUDIT_REPORT_2026-05-28.md for third-party verification.

Audit status of accompanying QA package

The synthetic double-slit reference benchmark passes the preregistered phase and visibility gates. The accompanying optics-null QA package includes a scoped publication gate that excludes generated summaries, templates, and other non-physical artifacts from aggregate null testing. Under the designated practical profile, the example null inputs pass the publication gate. These QA results should be interpreted as package-level verification, not as experimental validation of SFT.

APPENDIX

Optics Null Tests (Pre–Double-Slit) — SFT QA Protocol

Purpose. This document defines the null (baseline) tests that must pass before running the Double-Slit experiments in the SFT validation pipeline. It standardizes acceptance thresholds, artifacts, and corrective actions.

1) Placement & Scope

- Place this step immediately before the Double-Slit phase/visibility runs. If any null test fails, do not proceed to Double-Slit until a correction is documented.
- Scope: optical signals that should be zero (or constant) under baseline conditions: dark current / background, phase offset at $S_0=0$, alignment residuals, camera bias, environmental monitors (temperature, vibration) after detrending.

2) What qualifies as a null signal

A channel is a null signal if, by design, its expected mean is 0 and it should exhibit no systematic drift versus a control variable (time, S_0 , etc.). Examples:

- Dark intensity at a blocked detector.
- Phase baseline with $S_0=0$ (or far below threshold).
- Differential sensor with opposite arms balanced.
- Background-subtracted visibility in a no-interference configuration.

3) Statistical tests & gates

For each numeric column marked as a null signal:

A) Mean $\approx$ 0 test (two-sided): compute the sample mean m , standard error se ; accept if the 95% CI $[m-1.96\cdot se, m+1.96\cdot se]$ contains 0 OR if $p\text{-value}>0.05$.

B) Drift $\approx$ 0 test against control x (time, S0, ...): fit $y = a \cdot x + b$ (OLS); accept if $p\text{-value}(|a|) > 0.05$.

C) Optional physical bounds: if the quantity is bounded (e.g., visibility $\in [0,1]$), flag values outside the range; clipping is allowed only at the reporting stage, not on raw data.

4) Multiple-testing control (FDR)

When many null channels are checked, control the false discovery rate via Benjamini-Hochberg (BH) at $q=0.05$ on the family of p-values (means and slopes separately). Any channel marked significant after BH counts as FAIL.

5) Manifest schema & default thresholds

You may use a JSON manifest to keep thresholds frozen (aligned with SFT's α -in policy). Global defaults and per-channel overrides are supported.

```
{
  "mean_p_min": 0.05,
  "slope_p_min": 0.05,
  "fdr_q": 0.05,
  "bounds": { "visibility": [0.0, 1.0] },
  "saturation_gap_max": 0.10,
  "channels": {
    "phase_null": {"mean_p_min": 0.05, "slope_p_min": 0.05},
    "I_dark":      {"abs_mean_max": 0.01}
  },
  "roles": {
    "predictors": ["time", "S0"],
    "null_signals": ["phase_null", "I_dark", "bg_residual"]
  }
}
```

6) Running the tests

Use the provided script (or your own) to evaluate all CSV files and emit PASS/FAIL summaries and artifacts.

```
# Example CLI
python null_tests.py --inputs data/*.csv --manifest manifest_null.json
--out artifacts/
```

```
# Optional explicit roles per file
python null_tests.py --csv dark_run.csv --x time --y I_dark --manifest
manifest_null.json
```

```
# Expected outputs
# - optics_null_inventory.csv (paths, sizes, SHA-256)
# - optics_null_summary_per_column.csv (mean/CI, slope, p-values,
PASS/FAIL)
# - optics_null_summary_per_file.csv (aggregated PASS/FAIL per file)
```

```
# - plots/ worst_* .png (top-3 failing channels with OLS fits)
# - optics_null_overall.json (pointers to all artifacts)
```

7) Interpreting results & gating decisions

Outcome	Cause (typical)	Action	Proceed to Double-Slit?
PASS (all null tests)	Baseline is clean	Document artifacts; lock manifest commit/seed	Yes
FAIL — mean $\neq$ 0	Offset / mis-zero / background	Re-zero baseline; document correction; re-run null tests	No
FAIL — slope $\neq$ 0	Drift vs time/S0; ROI not fixed	Stabilize ROI/gains; add drift correction; re-run	No
FAIL — bounds	Saturation or mis-scaled units	Fix scaling/ROI; clip only for reporting; re-run	No

8) CI integration

Run null tests automatically on every data push (or nightly). Mark the job as PASS only if:

- All null channels pass mean/slope tests after FDR.
- No bounds violations on bounded quantities.
- Artifacts (CSV/JSON/PNG) are uploaded and checksummed.

Recommended: emit a single PASS/FAIL badge and attach the three worst-case plots for quick triage.

9) Worked example (synthetic)

A synthetic run with 12 null channels and 2 predictors produced:

- optics_null_summary_per_column.csv — all null channels had $p > 0.25$ on means and $p > 0.2$ on slopes; BH-FDR($q=0.05$) accepted all.

- optics_null_summary_per_file.csv — all files PASS.
 - No bounds violations; visibility stayed within [0,1].
- Proceed to Double-Slit: YES.

Appendix A — Statistical formulas

Mean test: $m = (1/n)\sum y_i$, $se = s/\sqrt{n}$ with $s^2 = \sum (y_i - m)^2 / (n-1)$. 95% CI: $m \pm 1.96 \cdot se$ (normal approx.). Two-sided p-value from $z = m/se$.

Slope test: OLS fit $y = a \cdot x + b$; $se(a) = \sqrt{(\sigma^2 / \sum (x_i - \bar{x})^2)}$ with $\sigma^2 = \sum (y_i - \hat{y}_i)^2 / (n-2)$. Two-sided p from $t = a/se(a)$.

Benjamini–Hochberg FDR (q): sort $p_{(1)} \leq \dots \leq p_{(m)}$; find $k = \max\{i: p_{(i)} \leq (i/m) \cdot q\}$; reject $\{1..k\}$.

Appendix B — Minimal manifest template

```
{
  "mean_p_min": 0.05,
  "slope_p_min": 0.05,
  "fdr_q": 0.05,
  "roles": { "predictors": ["time", "S0"], "null_signals":
["phase_null", "I_dark"] }
}
```

The reproducibility and verification choices used in this benchmark follow standard practice in computational science and scientific auditing.

References (reproducibility / verification standards)

1. R.D. Peng, Science 334, 1226 (2011). <https://doi.org/10.1126/science.1213847>
2. V. Stodden, P. Guo, Z. Ma, PLoS ONE 8, e67111 (2013). <https://doi.org/10.1371/journal.pone.0067111>
3. NIST, FIPS 180-4: Secure Hash Standard (SHS) (2015). <https://doi.org/10.6028/NIST.FIPS.180-4>
4. P.J. Roache, Verification and Validation in Computational Science and Engineering (Hermosa Publishers, 1998).
5. I.B. Celik, U. Ghia, P.J. Roache, et al., J. Fluids Eng. 130, 078001 (2008). <https://doi.org/10.1115/1.2960953>